# Casts and Type Conversions

What the compiler converts for you and where it loses information, why the C-style cast is a trap, and the named casts: `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast` and `bit_cast`. Assumes Types, Initialization and Constants.

## What the compiler converts for you

You do not have to write a cast for most conversions: the compiler converts between arithmetic types by itself. These **implicit conversions** follow fixed rules, and the surprises come from not knowing them.




In an expression both operands are converted to the higher of their two types: int + double is a double.
Signed and unsigned of the same size: the signed value becomes unsigned.
Storing a result back into a smaller type truncates it, silently.

The ladder of arithmetic conversions: operands climb it, never descend.

Compiled results of the classic surprises:

| EXPRESSION | RESULT | WHY |
|---|---|---|
| `-1 < 1u` | **false** | the signed value is converted to unsigned and becomes huge |
| `uint8_t(200) + uint8_t(100)` | 300, stored back as 44 | small types are promoted to `int` before arithmetic; storing back wraps |
| `7 / 2` | 3 | integer division; `7 / 2.0` is 3.5 |
| `int g = 3.7;` | 3 | conversion to an integer truncates toward zero (`-3.9` becomes -3) |
| `char` 0xFF widened to `int` | -1 (signed `char`) | whether plain `char` is signed is implementation-defined |

- **Never compare signed with unsigned** (`-Wsign-compare`); use `std::ssize` or cast deliberately.
- Single-argument constructors are implicit conversions too: mark them `explicit` (see explicit and conversion operators).

---

## Narrowing and braces

A conversion that can **lose information** is called **narrowing**: double to int, a wide integer to a narrower one, a value that does not fit. With `=` it compiles quietly. With braces the language requires the compiler to complain, so braces are a free safety check.

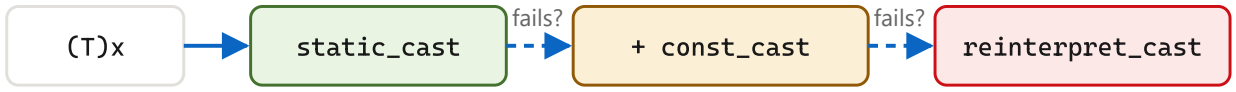
```
// compiles: a is 3, the .7 is lost without a word
int a = 3.7;
// int b{3.7};             // ERROR: narrowing double → int
// unsigned char c{300};   // ERROR: 300 does not fit in an unsigned char
unsigned char d{200};      // fine: a constant that fits
float f{0.1};              // fine: a constant double that fits in a float
```

| CONVERSION IN BRACES | ALLOWED? |
| --- | --- |
| a constant that fits the target (`unsigned char c{200}`) | yes |
| a constant that does not fit (`unsigned char c{300}`) | no: compile error |
| floating point to integer (`int b{3.0}`) | no: even when the value would fit |
| a wider integer variable to a narrower one | must be diagnosed; some compilers only warn, so turn warnings on |
| integer to `double` where every value fits | yes |

> HABIT Initialize with braces. When you really want the loss, say so with a cast (next sections), so the next reader knows it is intentional.

## The C-style cast and why to avoid it

C has one cast, `(T)x`. In C++ it is a trap: it tries the C++ casts in a fixed order and uses the **first one that compiles**, so a small mistake silently becomes a dangerous conversion.




The first that compiles wins. You do not see which one you got.

A C-style cast can quietly drop const or reinterpret raw bytes.

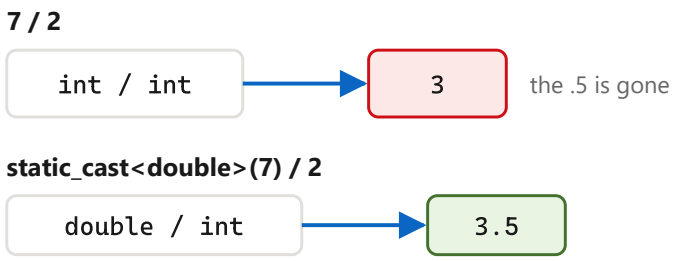
```
const int k = 5;
// compiles: silently removed the const (a const_cast)
int* p = (int*)&k;
// compiles: reinterpret the same address as another type
char* q = (char*)p;
// fine, but a static_cast says so: static_cast<double>(7)
double d = (double)7 / 2;
```

C++ replaces it with four named casts. Each states its intent, and you can search for it (`grep static_cast`). A C-style cast never turns into `dynamic_cast`.

| CAST | USE FOR | DANGER |
|---|---|---|
| `static_cast<T>(x)` | well-defined conversions you can see are fine | low |
| `dynamic_cast<T>(x)` | a checked downcast in a class hierarchy | runtime cost |
| `const_cast<T>(x)` | add or remove `const` | writing to a truly const object is undefined |
| `reinterpret_cast<T>(x)` | reinterpret raw bits | very high, rarely correct |

## `static_cast` : the everyday cast

`static_cast` does the conversions the compiler already knows how to do, but **explicitly**. It is checked at compile time and refuses nonsense such as int to pointer.




Convert one operand first and the whole division is done in floating point.

```
// avoids integer division
double avg = static_cast<double>(total) / count;

// -3: truncates toward zero, on purpose
int n = static_cast<int>(-3.9);
void* v = &n;
// void* back to the real type
int* p = static_cast<int*>(v);

Base* b = getBase();
// downcast: YOU promise it is a Derived
Derived* d = static_cast<Derived*>(b);
```

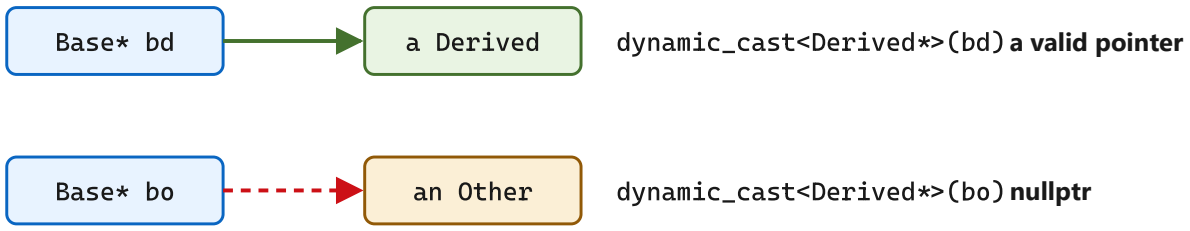
| USE | EXAMPLE |
| --- | --- |
| number to number | `static_cast<double>(i)`, `static_cast<int>(x)` |
| `void*` back to a typed pointer | `static_cast<Widget*>(userData)` |
| derived to base, base to derived | the base-to-derived direction is unchecked |
| enum to integer and back | `static_cast<int>(level)` |
| call an `explicit` conversion | `static_cast<std::string>(sv)` |

> **A DOWNCAST IS A PROMISE** If `b` is not really a `Derived`, the behavior is undefined and nothing checks it. When you are not sure, use `dynamic_cast`.

---

## `dynamic_cast`: checked downcasts

`dynamic_cast` asks the object what it **really is** at run time. If the answer is not the type you asked for, the cast fails safely instead of lying.




The check looks at the real object, not at the pointer type.

```
// nullptr if base is not a Derived
if (auto* d = dynamic_cast<Derived*>(base)) {
    // only reached when it really is
    d->fetch();
}

try { auto& r = dynamic_cast<Derived&>(*base); }
// a reference cannot be null, so it throws
catch (const std::bad_cast&) { }
```

- A pointer cast returns `nullptr` on failure; a reference cast throws `std::bad_cast`. Compiled: the Derived object passed, the Other object gave null and, as a reference, `bad_cast`.
- It needs a **polymorphic** base: at least one `virtual` function. That is how the object carries its real type (see the vtable).
- It costs a lookup at run time. `typeid(*p) == typeid(Derived)` asks for an exact match instead.
- A chain of `if (dynamic_cast<A*> ... ) else if (dynamic_cast<B*> ... )` usually means a virtual function is missing.

## `const_cast`, `reinterpret_cast` and `bit_cast`

### `const_cast`: only for old APIs

```
// an old C API: takes char* but only reads
void legacy(char* s);
const char* msg = "hello";
// ok: legacy never writes through it
legacy(const_cast<char*>(msg));
```

It removes `const` from the **type**, not from the object. Writing through the result is only defined if the object was not `const` to begin with; writing to a truly `const` object (a string literal, a `const int`) is undefined behavior.

### `reinterpret_cast`: look at the bytes

**int x = 0x01020304; read it as bytes:**




On a little-endian machine the lowest byte comes first. Compiled: bytes[0] was 4.

reinterpret_cast does not convert anything; it reads the same memory as another type.

```
int x = 0x01020304;
// allowed: any object may be read as bytes
auto* bytes = reinterpret_cast<const unsigned char*>(&x);
// 4 on a little-endian machine
bytes[0];

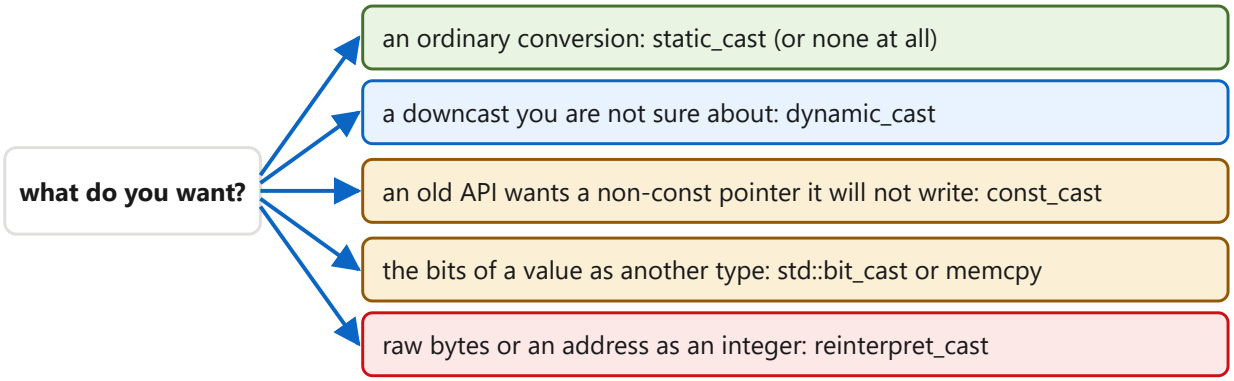
// pointer to integer and back is allowed
auto addr = reinterpret_cast<std::uintptr_t>(&x);
```

Reading an object as **another** type through such a pointer (an `int` as a `float`, for example) breaks the aliasing rules and is undefined. To reinterpret the bits of a value safely:

```
float f = 1.0f;
// always correct
std::uint32_t u1;  std::memcpy(&u1, &f, sizeof u1);
// C++20: the same, in one line
auto u2 = std::bit_cast<std::uint32_t>(f);
// both give 0x3f800000
```

---

## Which cast, if any?




Reach for them in this order; the lower the box, the rarer it should be.

**RULE** Many casts in one place mean a design problem: prefer virtual functions to `dynamic_cast` chains, templates to `void*`, and never write a C-style cast.

## Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **What the compiler converts for you** | C++ Primer | 2.1 Primitive built-in types | 61 to 69 |
| | C++ Primer | 4.11 Type conversions | 188 to 194 |
| **Narrowing and braces** | C++ Primer | 4.11 Type conversions | 188 to 194 |
| **The C-style cast and why to avoid it** | C++ Primer | 4.11 Type conversions | 188 to 194 |
| **static_cast: the everyday cast** | C++ Primer | 4.11 Type conversions | 188 to 194 |